UNIVERSIDAD POLITÉCNICA DE SINALOA

Ingeniería en Tecnologías de la Información e Innovación Digital

REPORTE TÉCNICO
EFECTO DOMINÓ

Simulación de Estrategias de Búsqueda BFS y DFS

Unidad 2: Algoritmos de Estrategias de Búsqueda

Actividad: EVD2.5-EVP2.5-EVC2.5

Materia: Inteligencia Artificial

Docente: Iliana Amabely Silva Hernandez

Integrantes del equipo: [Nombres pendientes]

Fecha: 9 de noviembre de 2026

ÍNDICE GENERAL

1. RESUMEN....3
1.1. Síntesis del Proyecto....3
2. INTRODUCCIÓN....3
2.1. Contexto Teórico....3
2.2. Planteamiento del Problema....4
2.3. Diagrama de caso de uso del caso de estudio....4
3. DESARROLLO....5
3.1. Arquitectura del Agente Inteligente....5
3.1.1. Identificación y Clasificación del Tipo de Agente....5
3.1.2. Matriz de Diagnóstico del Modelo PEAS/RASA....5
3.2. Propiedades del Entorno de Búsqueda....6
3.4. Diagrama de componentes del caso a desarrollar....6
3.5. Abstracción del Espacio de Estados....7
4. ALGORITMO DE BÚSQUEDA UTILIZADO....8
4.1. Descripción de la Estrategia Seleccionada....8
4.2. Formulación de Costos....9
5. CÓDIGO FUENTE....9
5.1. Implementación de Clases y Funciones....9
6. CAPTURA DE PANTALLA DE EJECUCIÓN....11
6.1. Evidencia del Sistema en Funcionamiento....11
6.2. Comparativa de Eficiencia entre Estrategias....14
7. CONCLUSIONES....15
7.1. Análisis y Cierre....15
7.2. Propuestas de Mejora....15
8.0. REFERENCIAS BIBLIOGRÁFICAS....16
9.0. APÉNDICES / ANEXOS....16

1. RESUMEN

1.1. Síntesis del Proyecto

El proyecto Efecto Dominó aborda el problema de determinar qué fichas serán alcanzadas desde un punto inicial y en qué tiempo discreto se propagará una reacción. La mesa se representa mediante una matriz de ocho filas y doce columnas, donde únicamente las fichas vecinas ortogonales pueden transmitir la caída. El objetivo es ofrecer un caso de estudio de búsqueda no informada mediante un agente basado en metas que interpreta el tablero, calcula resultados y los representa visualmente. La implementación utiliza Python, Tkinter y estructuras de la biblioteca estándar. BFS emplea una cola FIFO para obtener distancias mínimas y agrupar las fichas por niveles; DFS utiliza una pila LIFO para contrastar el orden de exploración y las rutas encontradas. La animación permite observar niveles, pausar y revisar pasos. La verificación confirmó 16 pruebas automatizadas, incluida la comparación exhaustiva de distancias en las 511 configuraciones no vacías de una matriz de 3 × 3. En el laberinto, BFS procesó 31 nodos y obtuvo una ruta de 9 conexiones; DFS procesó 26 y obtuvo 19 conexiones. Los resultados muestran la utilidad de BFS para propagación mínima, sin atribuirle una ventaja universal de tiempo de procesamiento.

2. INTRODUCCIÓN

2.1. Contexto Teórico

La inteligencia artificial estudia agentes que perciben un entorno y actúan para cumplir objetivos. En un agente basado en metas, la búsqueda permite examinar estados y transiciones antes de producir una solución (Russell y Norvig, 2021). El agente conoce el tablero completo y no aprende de datos históricos.

El tablero se interpreta como un grafo. BFS explora primero los vecinos más próximos mediante una cola FIFO: el primero que entra es el primero que sale. DFS profundiza mediante una pila LIFO: el último que entra sale primero. Ambas estrategias son no informadas porque no emplean una heurística. Los niveles BFS representan tiempos discretos con costos uniformes.

2.2. Planteamiento del Problema

Dada una matriz M de 8 × 12, con 1 para una ficha y 0 para una celda vacía, se requiere identificar el componente alcanzable desde una ficha inicial s. Si se selecciona una meta t, también se solicita una ruta. Para BFS, su longitud debe ser mínima. Una transición válida conecta fichas existentes arriba, abajo, a la izquierda o a la derecha; las diagonales y los huecos no transmiten la reacción.

La interfaz exige un inicio válido antes de ejecutar. Eliminar una ficha seleccionada elimina también su designación como inicio o meta. Durante la reproducción o pausa, la edición se bloquea; reiniciar o cargar un escenario cancela el temporizador. La meta opcional no detiene la animación completa: solo la comparación utiliza terminación al extraerla. El modelo es discreto, sin cálculo de fuerzas, gravedad ni colisiones.

2.3. Diagrama de caso de uso del caso de estudio

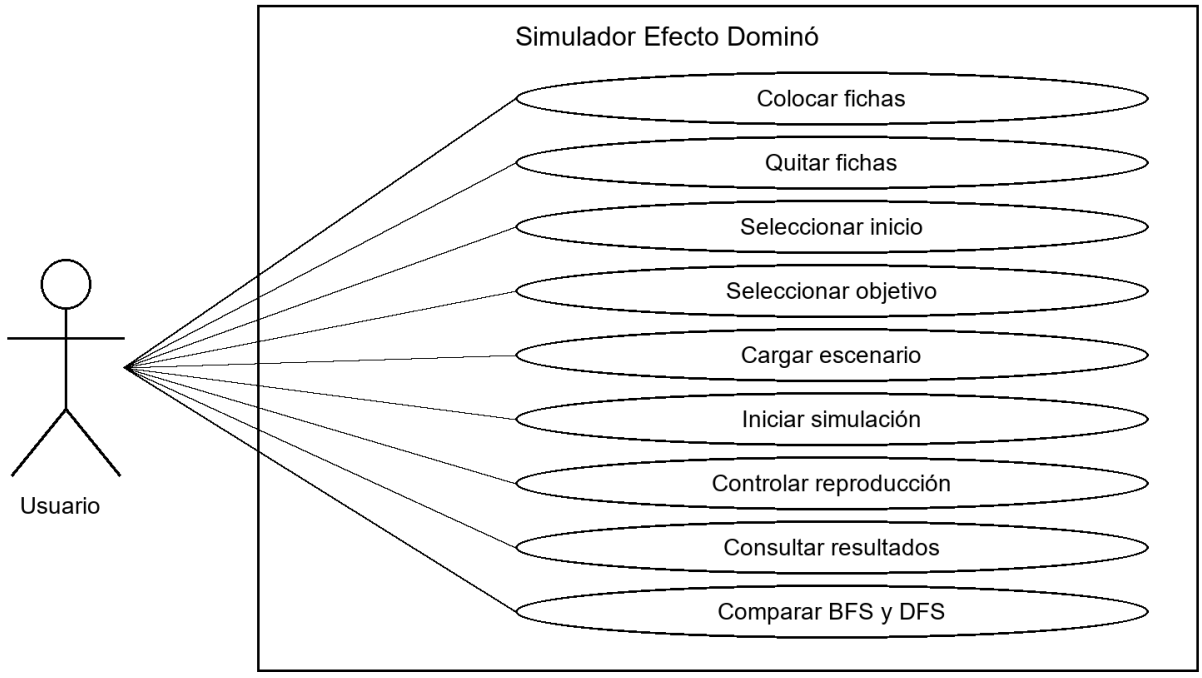


Figura 1. Casos de uso correspondientes a las operaciones de DominoApp.

Controlar reproducción comprende pausar, reanudar, avanzar, retroceder, reiniciar y ajustar la velocidad. Consultar resultados incluye el registro de exploración, las estadísticas y la ruta. Las asociaciones representan acceso del usuario a cada función; no implican una secuencia obligatoria.

3. DESARROLLO

3.1. Arquitectura del Agente Inteligente

La ejecución comienza en main.py, que crea DominoApp y entra en mainloop(). La clase de interfaz conserva la matriz, las selecciones y el estado de reproducción. algorithms.py recibe la matriz y el inicio, realiza la búsqueda y devuelve un objeto Result con orden, niveles, predecesores y duración. La lógica de búsqueda no importa Tkinter.

El flujo es lectura del escenario, validación, búsqueda, agrupación y presentación. prepare() calcula todo el componente antes de animarlo. tick() actualiza el grupo actual en doce fotogramas y programa el siguiente con after(). cancel_timer() evita que una ejecución anterior continúe después de reiniciar. Los botones de revisión reconstruyen estados desde los grupos precalculados, sin ejecutar nuevamente la búsqueda.

3.1.1. Identificación y Clasificación del Tipo de Agente

Se clasifica como agente basado en metas porque utiliza un modelo explícito de estados para determinar alcanzabilidad y, mediante BFS, un camino mínimo hacia una meta seleccionada. Los sensores y actuadores son abstracciones de software: lectura de entradas y actualización de la pantalla. No es un agente de aprendizaje, ni controla fichas físicas. La ejecución es deliberativa sobre una configuración fija.

3.1.2. Matriz de Diagnóstico del Modelo PEAS/RASA

Elemento	Aplicación al caso
Rendimiento	Exactitud de niveles, cobertura alcanzable, ruta mínima BFS y eficiencia de cálculo.
Entorno / ambiente	Mesa discreta de fichas y conexiones ortogonales.
Actuadores	Actualizar estados, dibujar fichas, rutas, conexiones y resultados.
Sensores	Leer matriz, inicio y meta; derivar conexiones por vecindad.

3.2. Propiedades del Entorno de Búsqueda

Propiedad	Justificación
Totalmente observable	La matriz y las selecciones están disponibles al iniciar. No hay fichas ocultas.
Determinista	Una misma configuración y orden fijo de vecinos producen el mismo recorrido y ruta.
Estático durante la ejecución	El cálculo es síncrono; la edición se bloquea durante la reproducción. Reiniciar abre una nueva ejecución.
Discreto	Hay 96 celdas posibles, estados de ocupación binarios y costos enteros por conexión.

El tiempo de CPU puede variar entre ejecuciones sin alterar estas propiedades. La orientación de la pieza dibujada y los puntos son recursos visuales: no modifican las aristas. La atenuación de fichas inaccesibles proviene del resultado de búsqueda, no de una medición física.

3.4. Diagrama de componentes del caso a desarrollar

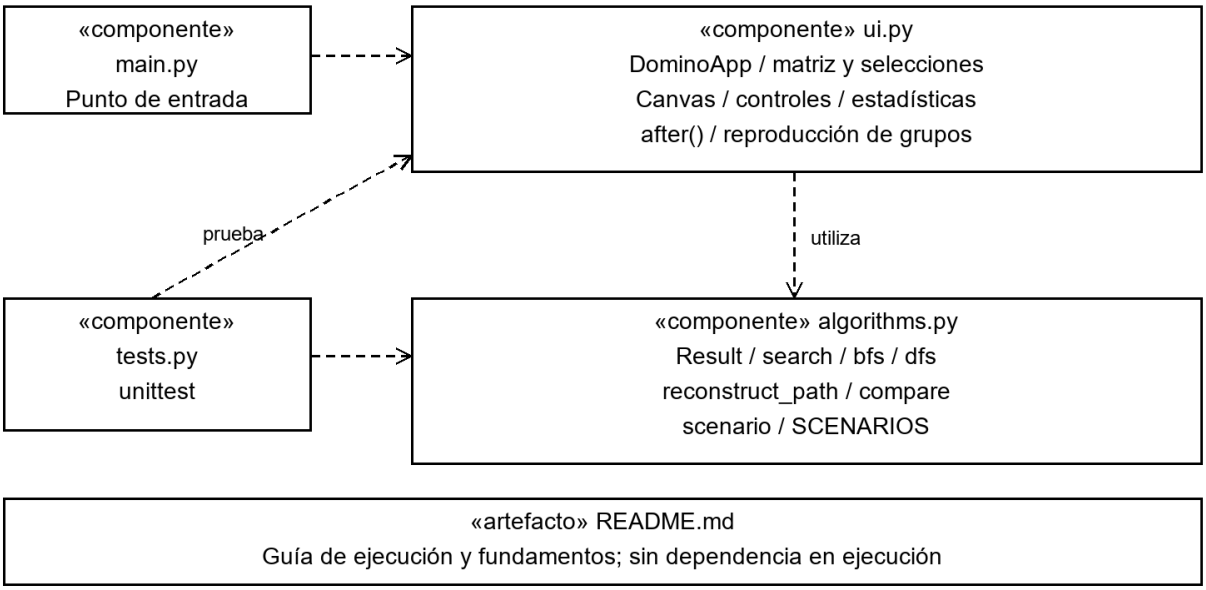


Figura 2. Componentes reales y dependencias. Las flechas de dependencia señalan el componente utilizado.

El modelo de mesa y la animación forman parte de ui.py; no existen módulos independientes para ellos. tests.py comprueba tanto las funciones de búsqueda como la reproducción y los datos de la tabla. README.md documenta el uso del proyecto.

3.5. Abstracción del Espacio de Estados

Sea G = (V, E) un grafo no ponderado. Cada posición ocupada es un vértice: V = {(r, c) | M[r][c] = 1}, con 0 ≤ r < 8 y 0 ≤ c < 12. Existe una arista entre u y v cuando ambas posiciones están ocupadas y |u.r − v.r| + |u.c − v.c| = 1. Las aristas son simétricas y no incluyen diagonales.

El estado inicial es s en V. Una acción selecciona un vecino válido; la transición cambia el nodo actual. La meta opcional es t en V. Para la simulación se busca el conjunto R de vértices alcanzables y la función de distancias d(s, v). El tiempo total de propagación es max{d(s, v) | v en R}; el nivel inicial vale cero.

Estructura	**Contenido y función**
frontier	Tuplas (nodo, padre, profundidad). deque en BFS; lista usada como pila en DFS.
visited	Conjunto de nodos procesados; evita explorar un nodo más de una vez.
queued	Conjunto de descubiertos. BFS impide insertar duplicados; DFS no aplica ese filtro.
levels / parents	Distancia BFS o profundidad DFS y predecesor del nodo procesado.
order / Result	Orden real de procesamiento y datos retornados a la interfaz.

Ejemplo: una cruz de cinco fichas con inicio en el centro. El centro tiene nivel 0 y sus cuatro vecinos nivel 1. Los cuatro comienzan su animación en el mismo grupo. El número de niveles es 2, pero el tiempo final discreto es 1. Una ficha separada por un hueco no pertenece a R y permanece sin caer.

El estado gráfico se conserva por separado en states. Cada ficha tiene un progreso de 0 a 1; la rotación representa hasta 90 grados. En BFS, groups reúne posiciones con igual distancia. En DFS, cada grupo contiene un único nodo y expresa exploración, no propagación mínima.

4. ALGORITMO DE BÚSQUEDA UTILIZADO

4.1. Descripción de la Estrategia Seleccionada

BFS inicia la cola con (s, None, 0). Extrae por la izquierda, omite nodos visitados y registra padre, distancia y orden. Después examina vecinos en orden arriba, abajo, izquierda y derecha. Los vecinos válidos no descubiertos se insertan al final con profundidad incrementada. collections.deque ofrece append() y popleft() para estas operaciones (Python Software Foundation, s. f.-a).

```
Validar matriz e inicio
Cola ← [(inicio, sin padre, 0)]; descubiertos ← {inicio}
Mientras la cola no esté vacía:
   Extraer (nodo, padre, nivel) por la izquierda
   Si ya fue visitado: continuar
   Registrar visitado, padre, nivel y orden
   Si se compara y nodo es meta: terminar
   Para cada vecino ortogonal válido no descubierto:
      Marcar descubierto
      Insertar (vecino, nodo, nivel + 1) al final
```

El pseudocódigo resume la rama BFS real. En DFS, frontier es una lista y se extrae con pop(). Los vecinos se insertan en orden inverso para priorizar la misma secuencia direccional. El padre se fija al procesar el nodo; las entradas duplicadas pendientes se descartan al extraerse si el nodo ya fue visitado. Por ello se conserva una exploración en profundidad sin bucles infinitos.

compare() ejecuta ambos métodos sobre la misma matriz y activa stop_at_target. Ambos terminan cuando extraen y procesan la meta, incluida en el conteo; si no hay meta o no es alcanzable, agotan la frontera. reconstruct_path() sigue los predecesores desde la meta y revierte la lista; devuelve una lista vacía si no hay ruta.

El costo temporal del recorrido es O(V + E). BFS utiliza memoria O(V); esta implementación DFS puede mantener entradas repetidas en la pila, por lo que su cota general es O(V + E). Como cada ficha tiene a lo sumo cuatro vecinos, E = O(V) en esta mesa y ambas usan espacio lineal. La animación y el dibujo se analizan aparte del cálculo.

4.2. Formulación de Costos

Se define g(inicio) = 0 y g(n) = g(padre) + 1. Cada arista vale una unidad de tiempo discreto. No se utiliza h(n), porque BFS es una estrategia no informada. El control de velocidad modifica la reproducción en milisegundos por grupo, pero no cambia g(n), las distancias ni las rutas.

La cola procesa los niveles en orden no decreciente. Cuando BFS descubre por primera vez un vecino, todos los caminos con menos conexiones ya tuvieron oportunidad de alcanzarlo. Con costos unitarios, esa primera distancia es mínima. DFS no mantiene este orden por capas, de modo que la ruta que encuentra puede ser más larga.

5. CÓDIGO FUENTE

5.1. Implementación de Clases y Funciones

Los siguientes fragmentos se extraen directamente de los archivos del proyecto. Result es el contenedor de resultados y DominoApp es la clase de interfaz. La ocupación de la mesa se inicializa mediante una lista de ocho filas independientes:

```
self.matrix = [[0]*12 for _ in range(8)]
```

Fuente: ui.py, líneas 18–18.

La selección de estructura, la extracción FIFO/LIFO y el registro de visitados se concentran en search():

```
frontier = deque([(start, None, 0)]) if algorithm == "BFS" else [(start, None, 0)]
parents, levels, order, visited = {}, {}, [], set()
queued = {start}
while frontier:
    node, parent, depth = frontier.popleft() if algorithm == "BFS" else frontier.pop()
    if node in visited:
        continue
    visited.add(node)
    parents[node], levels[node] = parent, depth
    order.append(node)
    if stop_at_target and node == target:
        break
```

Fuente: algorithms.py, líneas 28–39.

La parada por meta ocurre después de registrar el nodo. Esta ubicación explica por qué el conteo incluye a la propia meta. La validación previa rechaza matrices vacías o irregulares e inicios o metas sobre celdas sin ficha.

5.1. Implementación de Clases y Funciones (continuación)

La inserción incrementa el costo en una unidad y controla el descubrimiento de BFS. La duración registrada excluye validación inicial y animación:

```
        for other in neighbors if algorithm == "BFS" else reversed(neighbors):
            y, x = other
            if not (0 <= y < rows and 0 <= x < cols and matrix[y][x]) or other in visited:
                continue
            # BFS descubre al insertar; DFS fija el padre al extraer para profundizar.
            if algorithm == "BFS" and other in queued:
                continue
            queued.add(other)
            frontier.append((other, node, depth + 1))
    return Result(algorithm, order, levels, parents, perf_counter() - begin)
```

Fuente: algorithms.py, líneas 42–51.

La reconstrucción sigue únicamente predecesores obtenidos en la búsqueda:

```
def reconstruct_path(result, target):
    if target not in result.parents:
        return []
    path = []
    while target is not None:
        path.append(target)
        target = result.parents[target]
    return path[::-1]
```

Fuente: algorithms.py, líneas 62–69.

La comparación reutiliza search(); no contiene métricas predeterminadas:

```
def compare(matrix, start, target=None):
    """Misma terminación: extraer meta; sin meta, agotar el componente."""
    return tuple(search(matrix, start, name, target, True) for name in ("BFS", "DFS"))
```

Fuente: algorithms.py, líneas 72–74.

La interfaz agrupa por distancia antes de reproducir. En tick(), todas las posiciones del grupo reciben el mismo avance en un solo callback y luego se dibujan:

```
        self.frame += 1
        for node in self.groups[self.index]: self.states[node] = min(1, self.frame/12)
```

Fuente: ui.py, líneas 258–259.

```
        if self.frame >= 12:
            self.index += 1
            self.frame = 0
        self.job = self.after(max(1, int(self.speed.get()/12)), self.tick)
```

Fuente: ui.py, líneas 263–266.

6. CAPTURA DE PANTALLA DE EJECUCIÓN

6.1. Evidencia del Sistema en Funcionamiento

Las capturas corresponden a la aplicación ejecutada en Windows. La configuración de bifurcación contiene 15 fichas; el inicio está en la rama izquierda y la meta en el extremo superior.

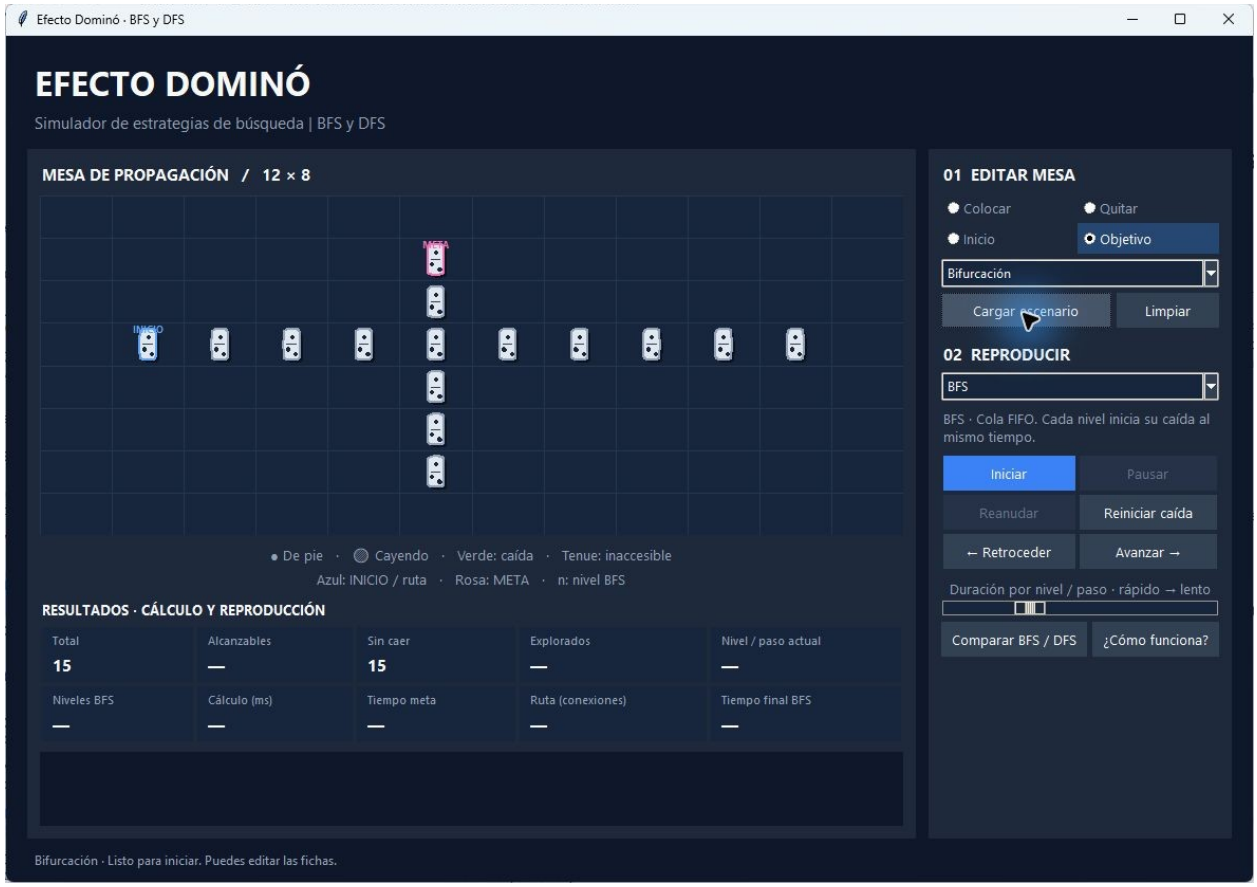


Figura 3. Interfaz principal con el escenario Bifurcación cargado.

Las herramientas del panel lateral permiten editar, elegir el algoritmo y controlar la reproducción. Las tarjetas inferiores separan el cálculo de la situación de las fichas durante la animación. Los bordes identifican inicio y meta.

6.1. Evidencia del Sistema en Funcionamiento (continuación)

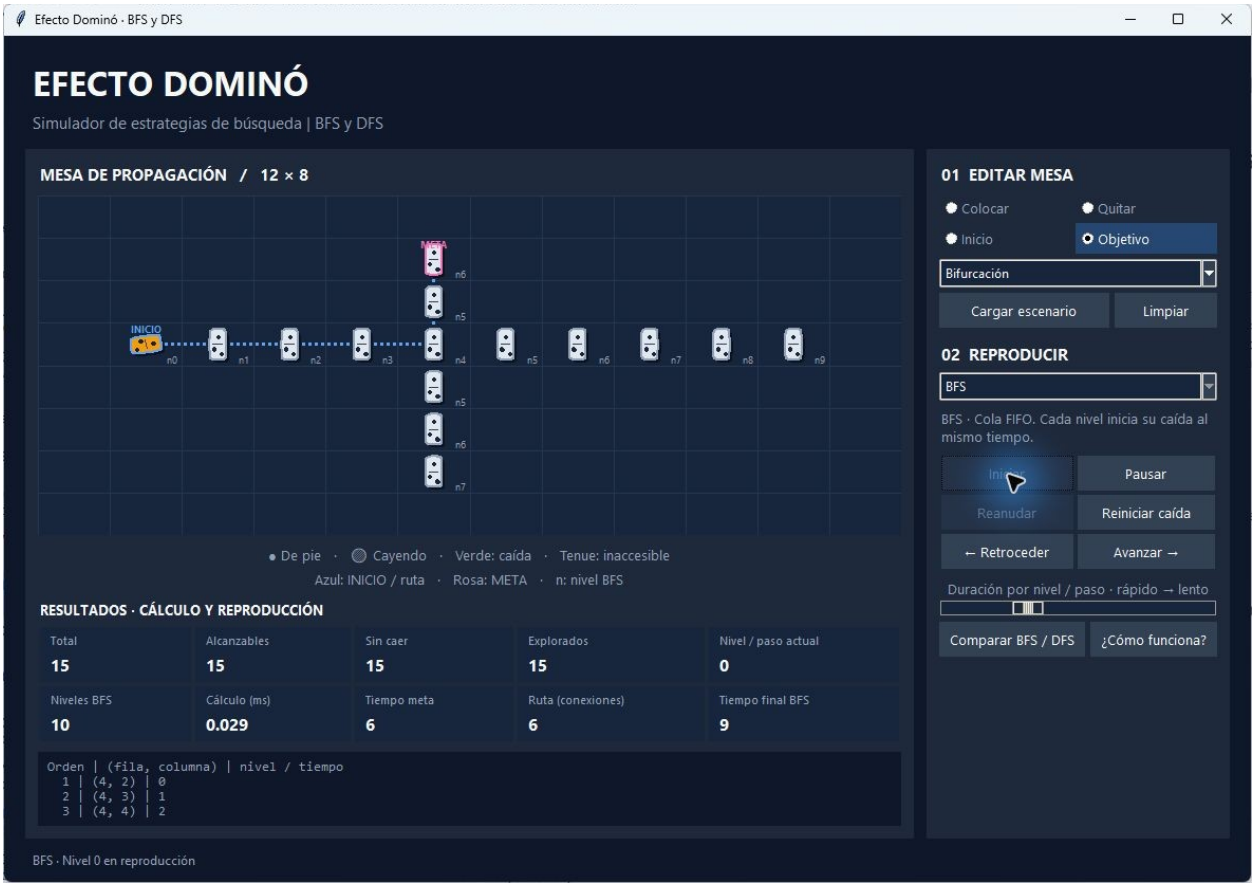


Figura 4. Simulación en curso: inicio ámbar y niveles calculados visibles.

La ficha inicial aparece inclinada y ámbar. Las demás conservan su posición de pie y muestran el nivel que recibirán al propagarse la reacción. Las estadísticas de alcanzables y explorados ya están calculadas, aunque la reproducción todavía no termina.

Los controles de pausa y revisión permiten detener la presentación sin recalcular las distancias. La simultaneidad por niveles se comprobó además en las pruebas fotograma a fotograma; una captura estática, por sí sola, no demuestra esa propiedad temporal.

6.1. Evidencia del Sistema en Funcionamiento (continuación)



Figura 5. Bifurcación completada: 15 alcanzables, 0 sin caer y tiempo final 9.

El cálculo completo obtiene 10 niveles, del 0 al 9. La meta está a 6 conexiones. Estos valores no dependen de la velocidad de animación.



Figura 6. Comparación real de la bifurcación con terminación en la meta.

En esta captura BFS procesa 9 nodos y DFS 7; ambas rutas tienen 6 conexiones. No contradice las 15 fichas del recorrido completo: la comparación se detiene en la meta. Los tiempos que aparecen en pantalla son mediciones de esa ejecución particular.

6.2. Comparativa de Eficiencia entre Estrategias

Se ejecutó compare() con el escenario Laberinto de dominós, sin modificar sus posiciones. El inicio interno es (3, 1) y la meta (3, 10), con índices desde cero. Se utilizó Python 3.13.17 en Windows. Los siguientes datos proceden de una ejecución registrada en evidencias/mediciones.json, independiente de la captura de bifurcación.

Algoritmo	Nodos	Ruta	Cálculo (ms)	Ruta mínima
BFS	31	9	0.0483	Sí
DFS	26	19	0.0381	No garantiza

BFS obtuvo 9 conexiones frente a 19 de DFS: diez conexiones menos. DFS procesó 26 nodos frente a 31 de BFS: cinco nodos menos antes de llegar a la meta. Por tanto, procesar menos nodos en este ejemplo no produjo una ruta mejor. La elección depende de la propiedad buscada; para niveles de propagación mínima corresponde utilizar BFS.

Los tiempos se obtienen con perf_counter() (Python Software Foundation, s. f.-b): son intervalos transcurridos dentro de search(), no tiempo exclusivo de CPU ni duración de la animación. Una muestra tan breve no constituye un estudio de rendimiento estadístico; cambia por carga del sistema, resolución del contador y calentamiento del intérprete. No se concluye que BFS siempre sea más rápido.

La ejecución de unittest confirmó 16 pruebas aprobadas. La prueba exhaustiva evalúa las 511 ocupaciones no vacías de una matriz 3 × 3 y cada ficha como inicio: 2 304 búsquedas. Contrasta las distancias con relajación independiente de aristas. Esta cobertura prueba todos esos casos pequeños, sin sustituir una demostración general de corrección.

Otra prueba inspecciona los doce fotogramas de cada nivel de la bifurcación: verifica idéntico progreso en todo el grupo, niveles anteriores completos y niveles posteriores sin iniciar. La prueba de la tabla envuelve compare() sin sustituir sus cálculos y verifica que los valores visibles coincidan con nodos, rutas y tiempos reales en dos escenarios.

7. CONCLUSIONES

7.1. Análisis y Cierre

La implementación resuelve el caso de estudio de propagación sobre una mesa discreta. BFS permite conocer el componente alcanzable, las distancias mínimas y una ruta mínima hacia una meta opcional. La separación entre cálculo y animación facilita explicar que el nivel es una propiedad del grafo, mientras que la velocidad de reproducción es una decisión de presentación.

Las verificaciones confirman una cola FIFO real, control de visitados y sincronización de las fichas del mismo nivel. La diferencia observada en el laberinto muestra que DFS puede llegar a la meta después de procesar menos nodos y, aun así, entregar un camino más largo. Esta comparación es válida porque ambas búsquedas usan la misma matriz y la misma condición de terminación.

El agente basado en metas representa una abstracción didáctica: percibe entradas del editor y actúa sobre la visualización. No aprende, no controla objetos reales y no modela interacciones mecánicas. Los resultados son reproducibles en orden y distancias; las duraciones de cálculo pueden variar. La batería de pruebas y las evidencias respaldan el funcionamiento documentado, dentro de esas limitaciones.

7.2. Propuestas de Mejora

Como trabajo futuro, podría incorporarse orientación direccional para que cada ficha solo transmita la caída hacia determinadas vecinas. Un modelo físico más preciso requeriría dimensiones, separación, fuerzas y colisiones, además de una validación distinta. Estas capacidades no forman parte de la versión actual.

También se propone exportar escenarios y resultados, admitir mesas mayores y comparar con otras estrategias. Si se introducen costos diferentes por conexión, se deberá sustituir la garantía de BFS por un algoritmo apropiado para grafos ponderados. Las mediciones futuras deberían incluir repeticiones, escenarios de tamaños controlados y resúmenes estadísticos, manteniendo separadas búsqueda y animación.

8.0. REFERENCIAS BIBLIOGRÁFICAS

Python Software Foundation. (s. f.-a). *collections — Container datatypes*. En Python 3.13 documentation. https://docs.python.org/3.13/library/collections.html

Python Software Foundation. (s. f.-b). *time — Time access and conversions*. En Python 3.13 documentation. https://docs.python.org/3.13/library/time.html

Python Software Foundation. (s. f.-c). *tkinter — Python interface to Tcl/Tk*. En Python 3.13 documentation. https://docs.python.org/3.13/library/tkinter.html

Russell, S. J., & Norvig, P. (2021). *Artificial intelligence: A modern approach* (4.ª ed.). Pearson. https://aima.eecs.berkeley.edu/newchap00.pdf

9.0. APÉNDICES / ANEXOS

A. Estructura y ejecución

Archivo	**Responsabilidad**
main.py	Entrada: crea DominoApp y ejecuta mainloop().
algorithms.py	BFS, DFS, rutas, comparación y cinco escenarios.
ui.py	Matriz, editor, dibujo, reproducción y estadísticas.
tests.py	16 pruebas de algoritmos y comportamiento de interfaz.
README.md	Instalación, controles y fundamentos del proyecto.

Requisitos: Python 3.11 o posterior con Tcl/Tk y sesión de escritorio en Windows. Abrir la carpeta Domino en VS Code, seleccionar el intérprete y ejecutar python main.py. Para comprobar Tkinter: python -m tkinter. Para las pruebas: python -m unittest tests -v. La aplicación no necesita paquetes externos, base de datos ni servidor.

B. Casos de prueba y evidencia de unittest

Grupo	Casos verificados	Resultado
SearchTests (7)	Cadena, bifurcación, aisladas, ciclos y niveles, desvío por obstáculo, comparación y rutas, inicio inválido.	7 aprobadas
PlaybackTests (3)	Pausa/reanudación y reinicio, pasos y cambio de algoritmo, fotogramas y finalización.	3 aprobadas
ScenarioTests (3)	Cinco escenarios y laberinto, meta ausente o inaccesible, meta igual al inicio.	3 aprobadas
DeliveryChecks (3)	Distancias exhaustivas, simultaneidad por fotograma, datos reales de tabla.	3 aprobadas

La siguiente transcripción procede del archivo de salida conservado en evidencias/unittest.txt; es evidencia textual del comando ejecutado, no una captura reconstruida de una terminal:

```
----------------------------------------------------------------------
Ran 16 tests in 1.935s

OK
```

El conjunto exhaustivo contiene 511 máscaras de ocupación y considera todos sus inicios ocupados. El total de casos internos es $9 \times 2^8 = 2\,304$. Los obstáculos, desconexiones y ciclos aparecen entre esas configuraciones. La prueba usa relajación de aristas como referencia independiente de la cola FIFO.

C. Trazabilidad y límites de la evidencia

Las imágenes bifurcacion.png, progreso.png, completada.png y comparacion.png son capturas de ventanas reales. casos_uso.png y componentes.png son diagramas explicativos elaborados a partir del código. mediciones.json conserva el intérprete y la muestra del laberinto; unittest.txt conserva la salida de pruebas. Los scripts de documentación están separados de los módulos de la aplicación.

Los archivos de la aplicación se conservaron sin modificaciones durante la elaboración del reporte. Las evidencias no incluyen un estudio físico ni una evaluación de rendimiento a gran escala. Los nombres del equipo deben completarse en la portada editable antes de la entrega académica.